



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

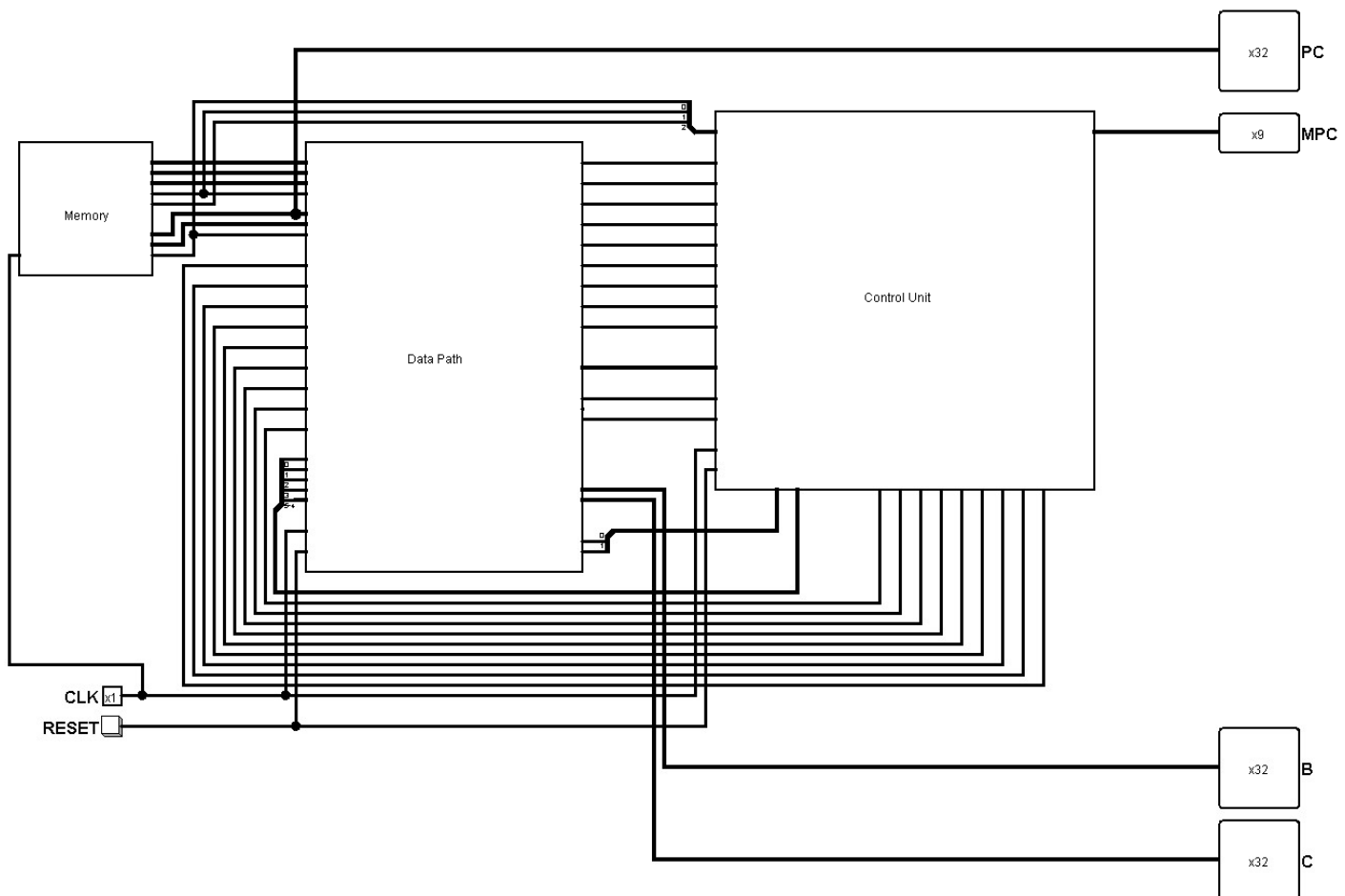
PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matías José

Proyecto final Arquitectura y Organización de las Computadoras: MIC-1

INTRODUCCIÓN:

El presente trabajo tiene como objetivo realizar el diseño y análisis de la MIC-1. Creada por Andrew S. Tanenbaum, esta arquitectura tiene el propósito de actuar como intérprete para ejecutar un conjunto de instrucciones basado en enteros denominado IJVM (Integer Java Virtual Machine). La MIC-1 fue diseñada para fines pedagógicos por lo que es una excelente herramienta de estudio, aunque contiene algunas deficiencias a nivel rendimiento que se expondrán a lo largo de este trabajo.

En las siguientes secciones de este proyecto desarrollaremos con detalle los componentes de la MIC-1 y los representaremos usando la herramienta Logisim, que sirve para diseñar y simular circuitos lógicos digitales.



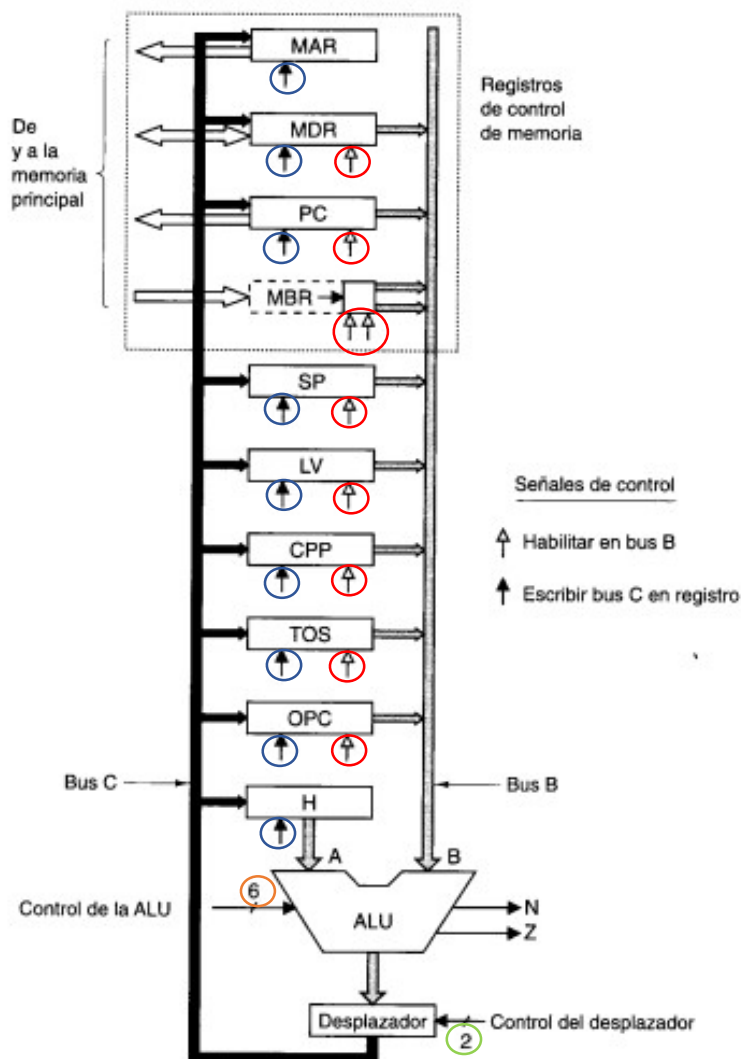
MIC-1 main - Logisim



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José

CAMINO DE DATOS



El Camino de Datos es la parte de la CPU que contiene la ALU(Unidad Aritmética Lógica), sus registros, entradas y salidas, formando la trayectoria principal por donde fluye la información. Su operación básica consiste en seleccionar uno o dos de los registros de 32 bits de la microarquitectura, hacer que la ALU opere con ellos (por ejemplo, sumándolos) y almacenar luego el resultado.

Para realizar estas operaciones, la ALU necesita dos entradas de datos: una izquierda (A) y una derecha (B). La entrada derecha está conectada al Bus B, el cual puede recibir el contenido de casi todos los registros (MDR, PC, MBR, SP, LV, CPP, TOS, OPC). Sin embargo, la entrada izquierda está conectada únicamente a un registro de retención dedicado llamado H. Una vez que los datos son procesados, la salida de la ALU alimenta al desplazador y luego pasa al Bus C, desde donde el resultado puede escribirse en

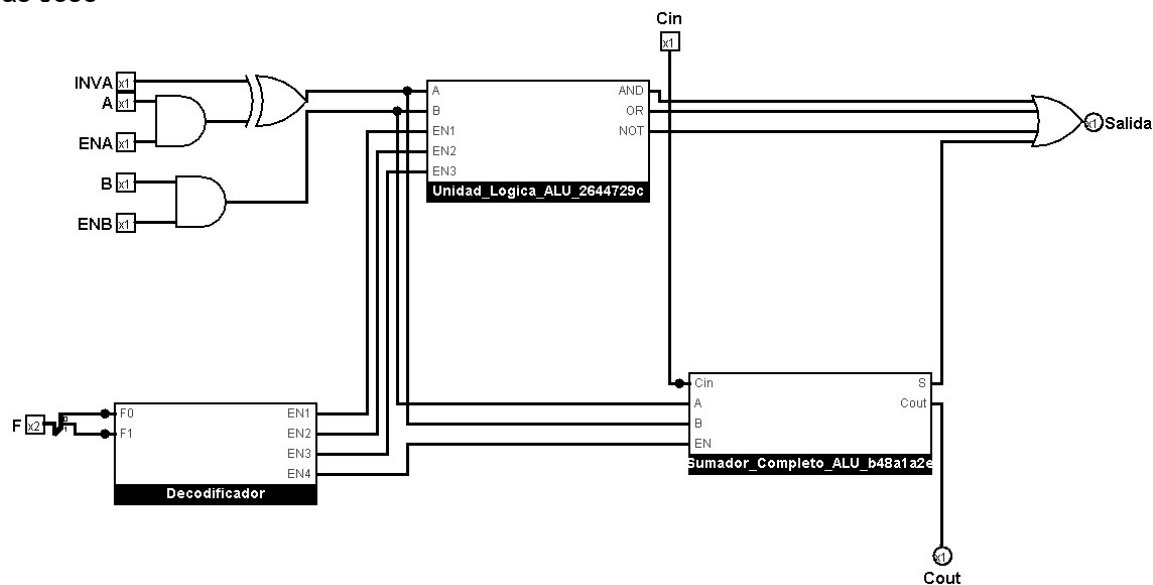
uno o más registros al mismo tiempo.

Al analizar este diseño, encontramos una clara deficiencia de rendimiento: al momento de querer realizar una operación con dos operandos, se desperdicia un ciclo de reloj entero simplemente para guardar el primer valor en el registro H. Solo después de este paso previo, el dato puede ingresar a la ALU por la entrada A, mientras el segundo valor entra simultáneamente por el Bus B.

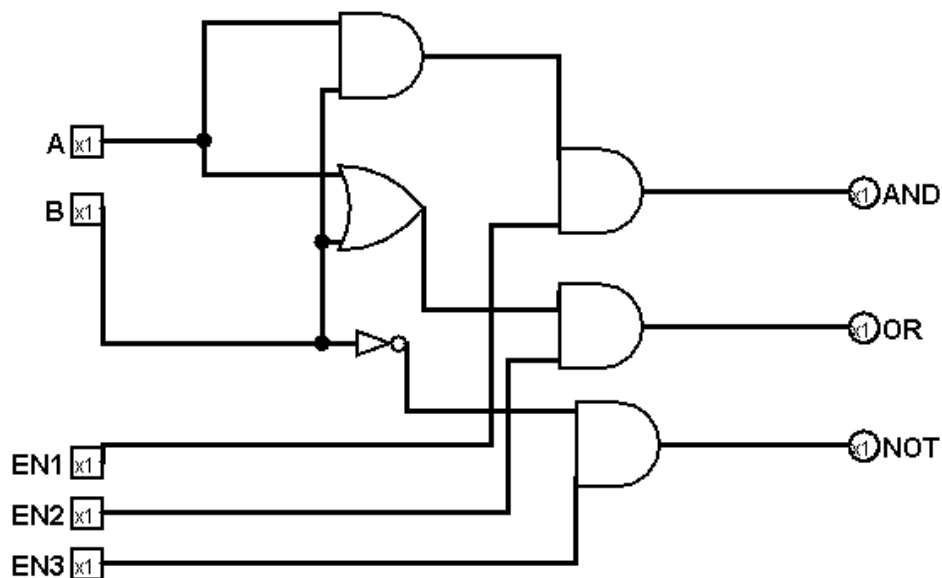


**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José



ALU de 1 bit - Logisim

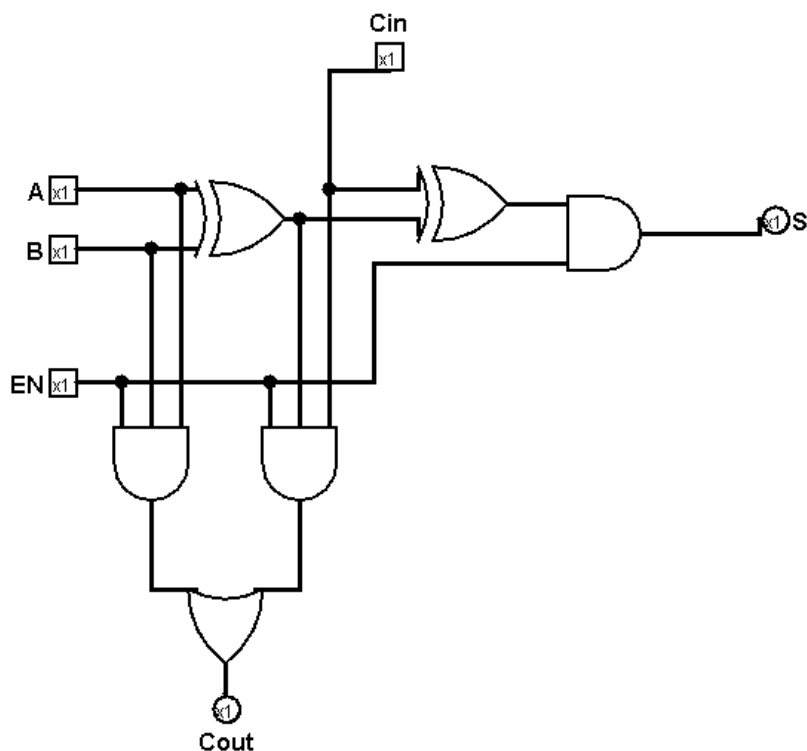


Unidad Lógica ALU - Logisim

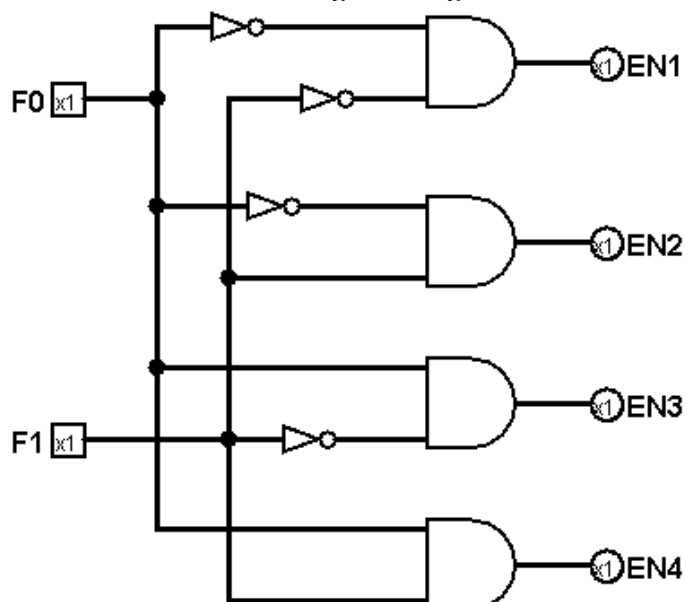


ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José



Sumador Lógico - Logisim



Decodificador-Logisim

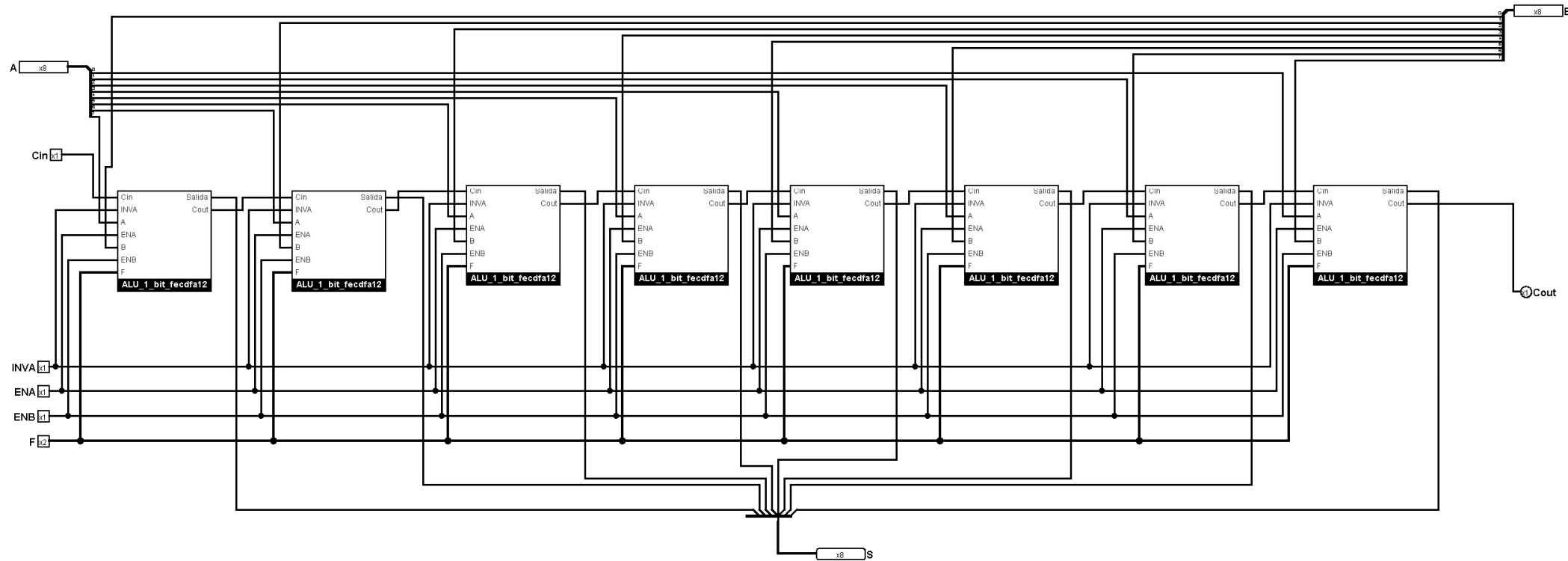
**PROGRAMADOR UNIVERSITARIO
LICENCIATURA EN INFORMÁTICA
INGENIERÍA EN INFORMÁTICA**

Facultad de Ciencias Exactas y Tecnología
Universidad Nacional de Tucumán



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres
Matias José



ALU de 8 bits uniendo 8 ALU's de 1 bit- Logisim

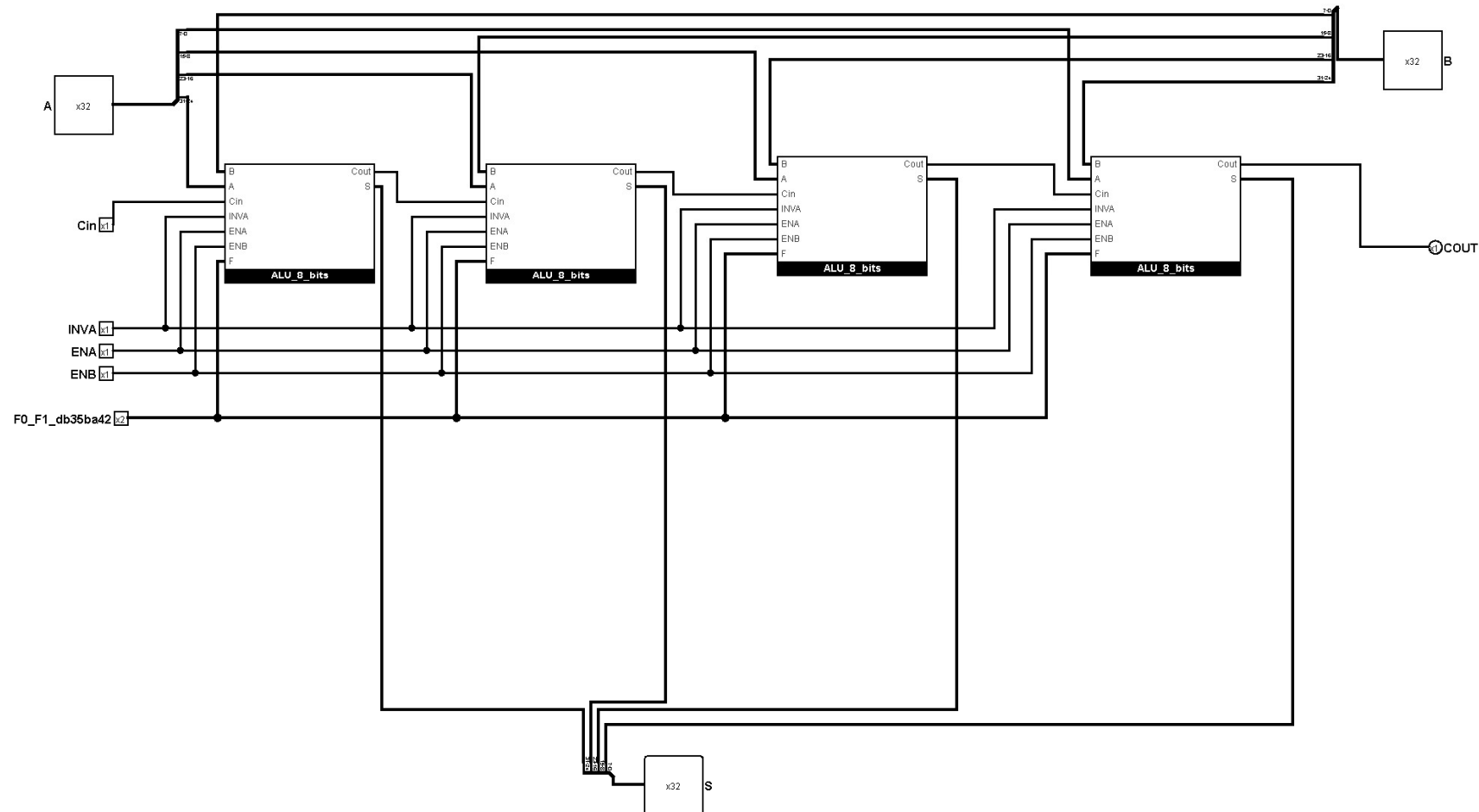
**PROGRAMADOR UNIVERSITARIO
LICENCIATURA EN INFORMÁTICA
INGENIERÍA EN INFORMÁTICA**

Facultad de Ciencias Exactas y Tecnología
Universidad Nacional de Tucumán



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres
Matias José



ALU 32 bits usando 4 ALU's de 8 bits - Logisim

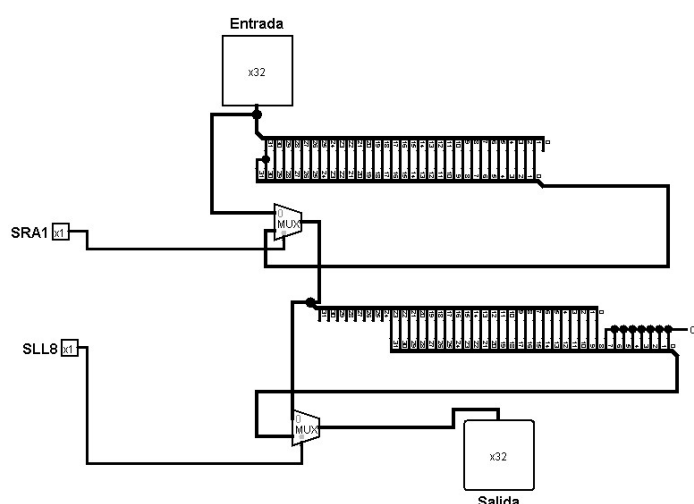


**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José

La función de la ALU está determinada por seis líneas de control. Estas son F0 y F1 para determinar el funcionamiento de la ALU, ENA y ENB para habilitar individualmente las salidas, INVA para invertir la entrada izquierda, e INC para forzar un acarreo de entrada en el bit de orden bajo, lo que en realidad suma 1 al resultado. Sin embargo, no todas las 64 combinaciones de las líneas de control de la ALU hacen algo útil. Algunas de las combinaciones mas interesantes son:

F0	F1	ENA	ENB	INVA	INC	FUNCION
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\bar{A}$
1	0	1	1	0	0	$\bar{B}$
1	1	1	1	0	0	A+B
1	1	1	1	0	1	A+B+1
1	1	1	0	0	1	A+1
1	1	0	1	0	1	B+1
1	1	1	1	1	1	B-A
1	1	0	1	1	0	B-1
1	1	1	0	1	1	-A
0	0	1	1	0	0	A AND B
0	1	1	1	0	0	A OR B
0	1	0	0	0	0	0
1	1	0	0	0	1	1
1	1	0	0	1	0	-1



Inmediatamente después de la ALU, los datos pasan por un desplazador antes de volcarse en el Bus C. Su función es realizar desplazamientos de bits a la izquierda o a la derecha sobre el resultado de 32 bits de la ALU. Este componente es controlado por dos señales independientes: SRA1 (Shift Right Arithmetic 1) y SLL8 (Shift Left Logical 8).

Si se activa SRA1, el desplazador corre todos los bits una posición hacia la derecha manteniendo intacto el bit de signo, lo cual equivale matemáticamente a dividir un número entero por dos. Por otro lado, si se activa SLL8, los bits se desplazan ocho posiciones hacia la

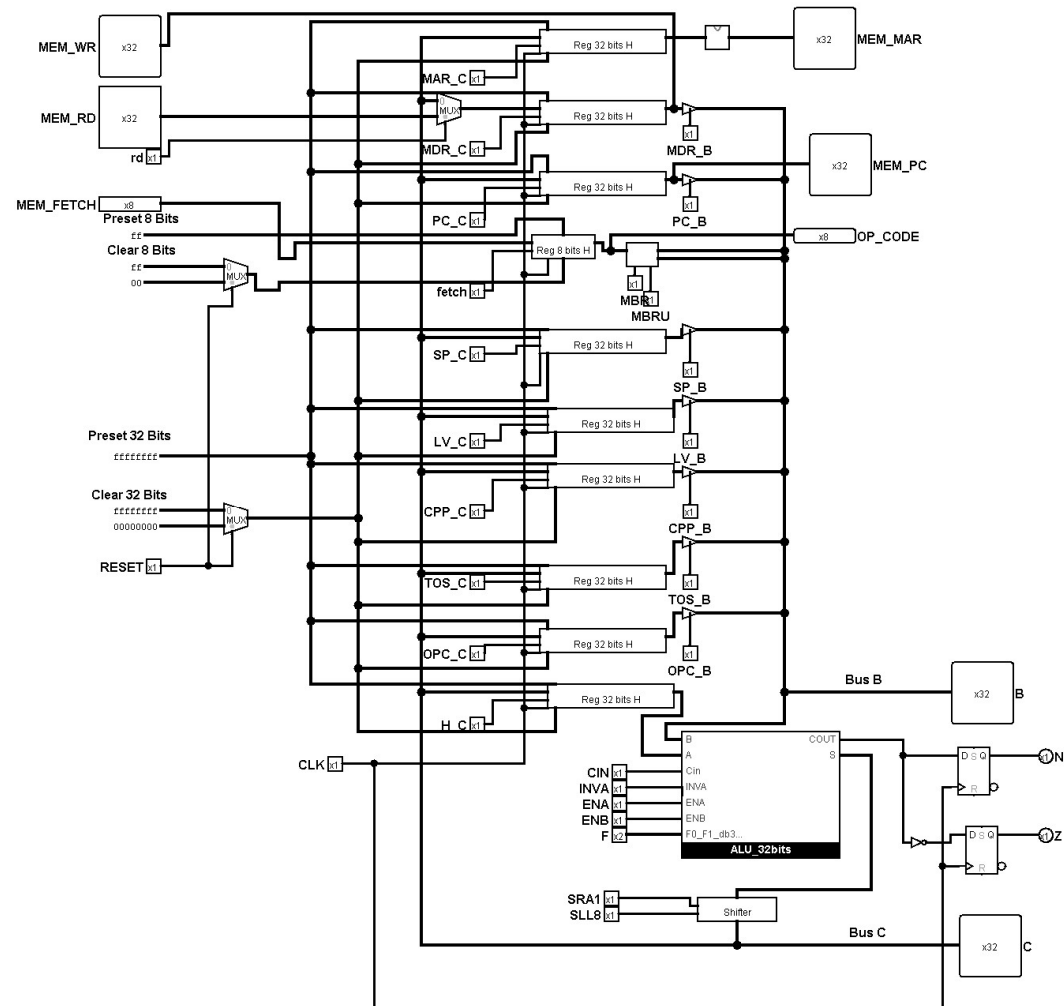
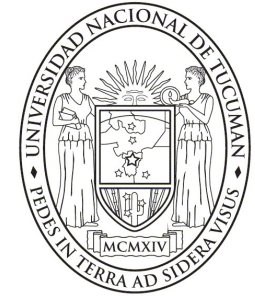
izquierda, relleno con ceros los espacios vacíos a la derecha. Si ninguna de las dos señales de control está activa, el desplazador simplemente actúa como un puente, dejando pasar el resultado de la ALU hacia el Bus C sin modificaciones.

**PROGRAMADOR UNIVERSITARIO
LICENCIATURA EN INFORMÁTICA
INGENIERÍA EN INFORMÁTICA**

Facultad de Ciencias Exactas y Tecnología
Universidad Nacional de Tucumán

**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres
Matias José



Data Path-Logisim

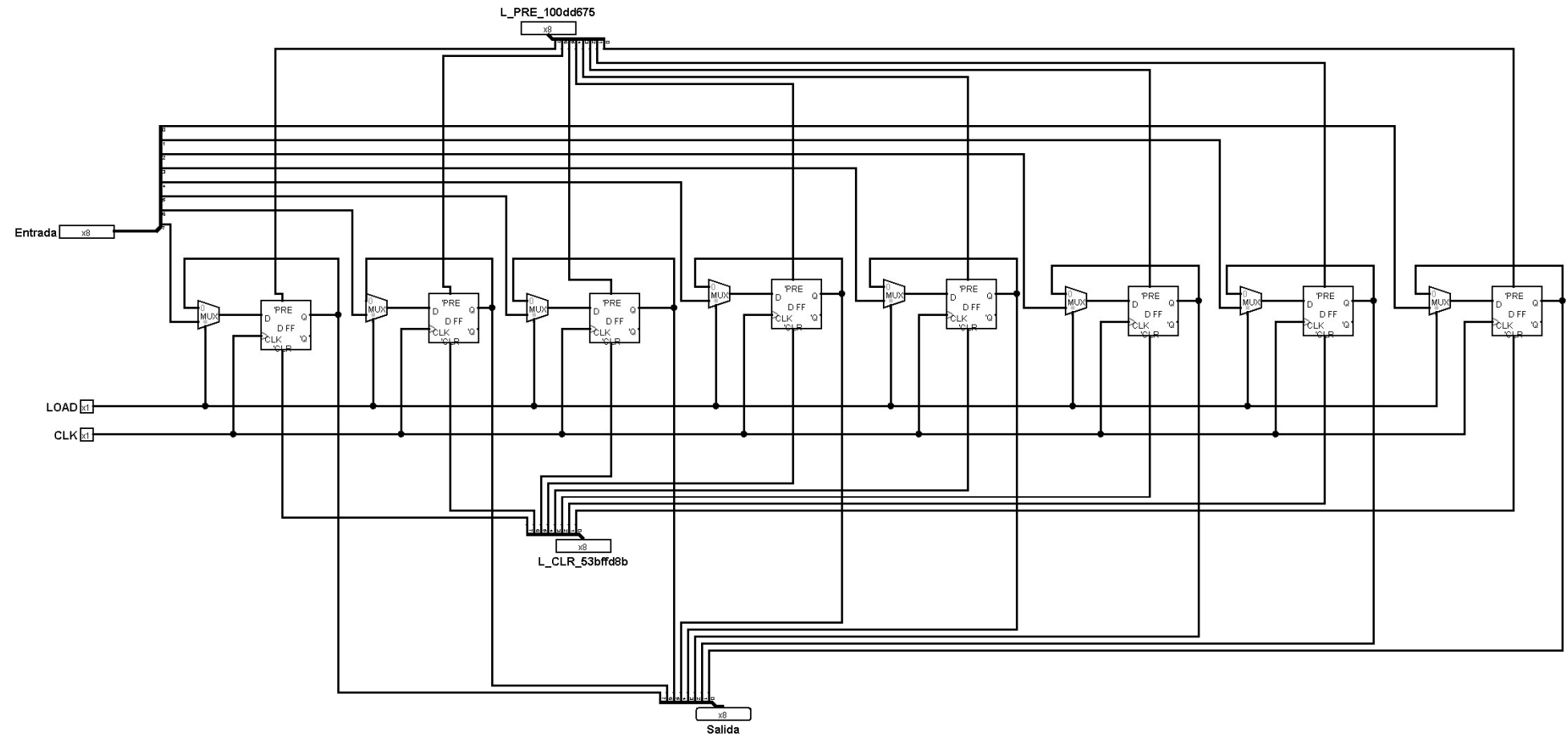
**PROGRAMADOR UNIVERSITARIO
LICENCIATURA EN INFORMÁTICA
INGENIERÍA EN INFORMÁTICA**

Facultad de Ciencias Exactas y Tecnología
Universidad Nacional de Tucumán



**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres
Matias José



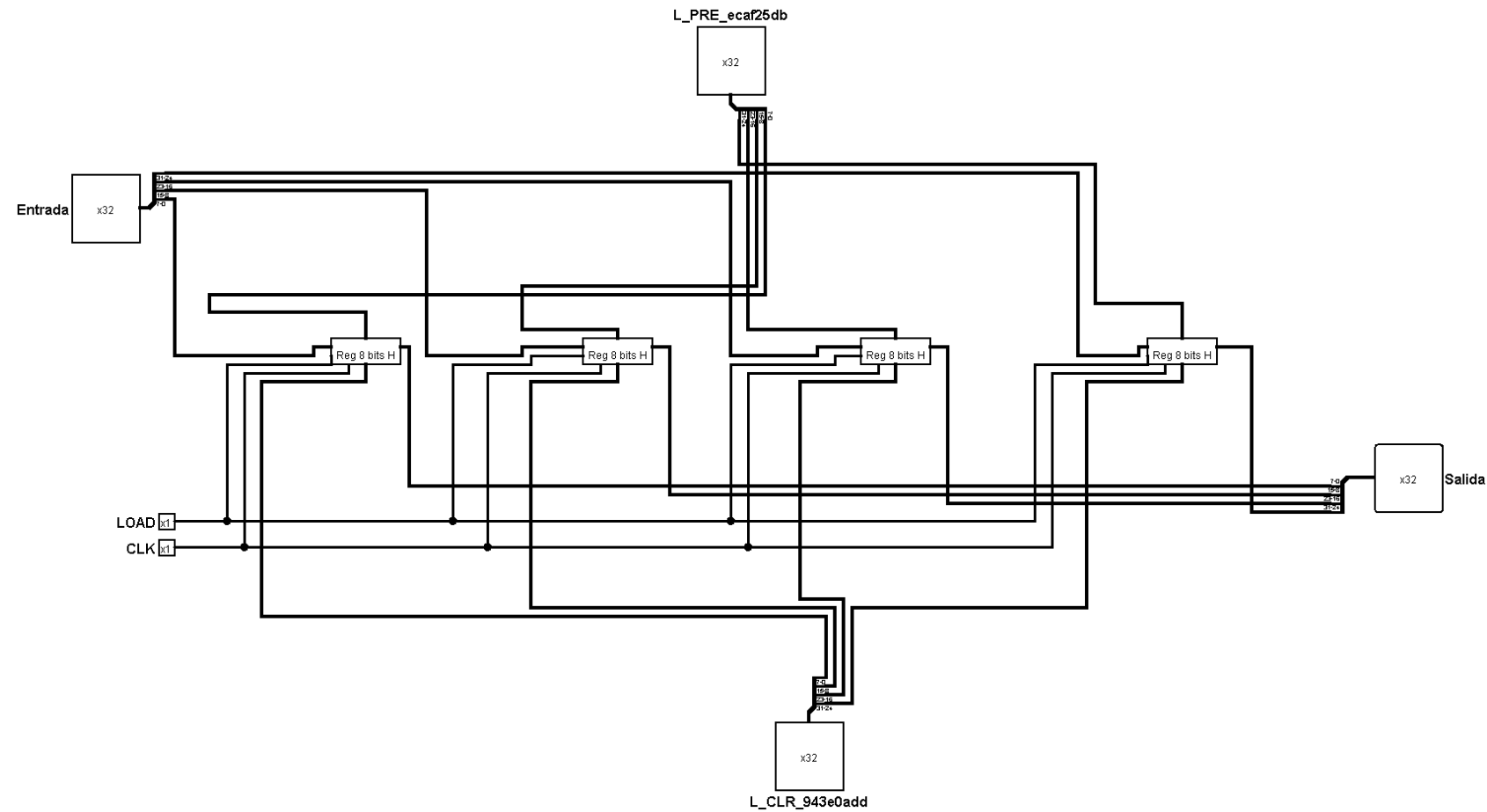
Registro 8 bits - Logisim

**PROGRAMADOR UNIVERSITARIO
LICENCIATURA EN INFORMÁTICA
INGENIERÍA EN INFORMÁTICA**

Facultad de Ciencias Exactas y Tecnología
Universidad Nacional de Tucumán

**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

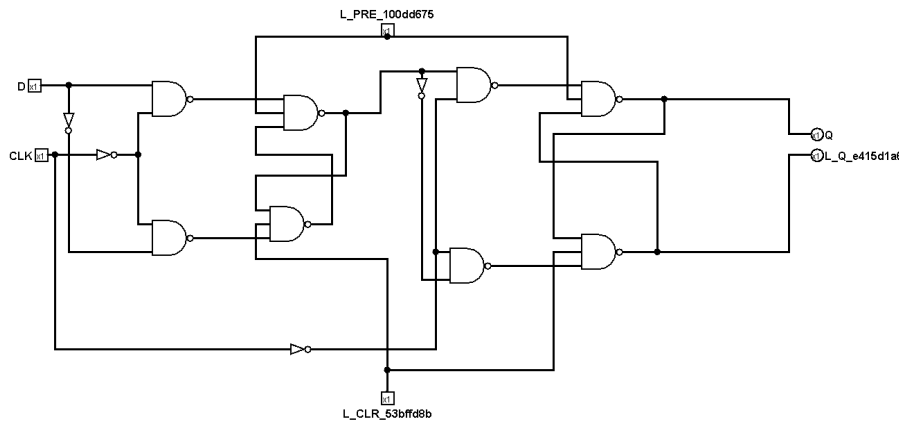
PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres
Matias José



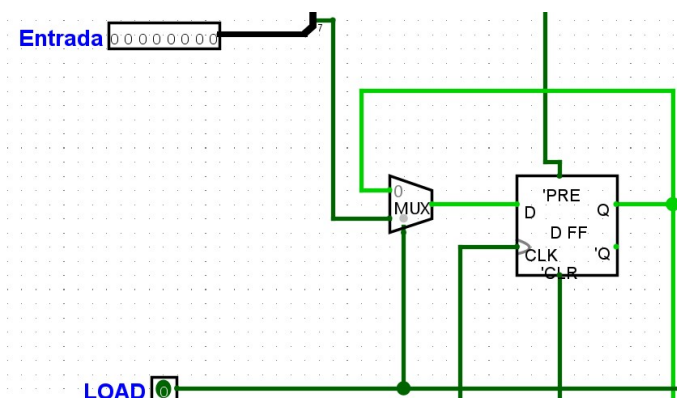
Registro de 32 bits a partir de 4 registros de 8 bits -Logisim

**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José

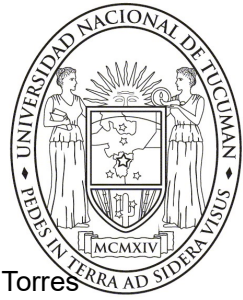


En el registro de 8 bits y, por ende, en el de 32, usamos flip-flops ascendentes. Estos son elementos capaces de guardar un bit. Tienen una entrada D, una salida Q, una entrada de reloj llamada CLK y otras entradas CLR y PRE. En este caso, como son con flanco ascendente, el dato se guarda en el momento en el que el CLK pasa de 0 a 1. Usamos flancos porque todas las partes del procesador deben cambiar de manera ordenada. Por ejemplo, el MIR, que veremos con más detalle en otra sección, puede cargar una microinstrucción en el flanco descendente mientras otros registros se actualizan en otro momento del ciclo. Así se evita que varios valores cambien al mismo tiempo y se mezclen datos viejos con datos nuevos. Además la señal CLR borra el contenido del registro y lo lleva a cero, si está implementado y la señal PRE fuerza los bits a uno.



La función del multiplexor, en el circuito lo vemos como MUX, es regular la carga en paralelo y retención del registro. Es necesario ya que la señal del reloj está oscilando de forma constante por lo que debemos regular cuando queremos que se cambie la información del flip-flop. Vemos que la entrada 1 del MUX está conectada a la entrada principal (lo que viene del bus C), la entrada 0 está conectada a la propia salida del flip-flop y el selector está unido a LOAD. Cuando LOAD está en 1, se habilita la entrada 1 del MUX y deja pasar al flip-flop el nuevo dato del bus C. En cambio, cuando LOAD está en 0 el MUX se cambia a la entrada 0 y vuelve a ingresar al flip-flop el dato que ya tenía anteriormente.

Es decir, la secuencia es la siguiente: el multiplexor elige entre mantener el dato anterior o tomar uno nuevo del bus C. Esa elección depende de LOAD. Después, cuando llega el flanco del reloj, los 32 flip-flops guardan en paralelo el valor elegido. Si LOAD vale 0, el registro retiene su contenido; si LOAD vale 1, carga el nuevo dato.

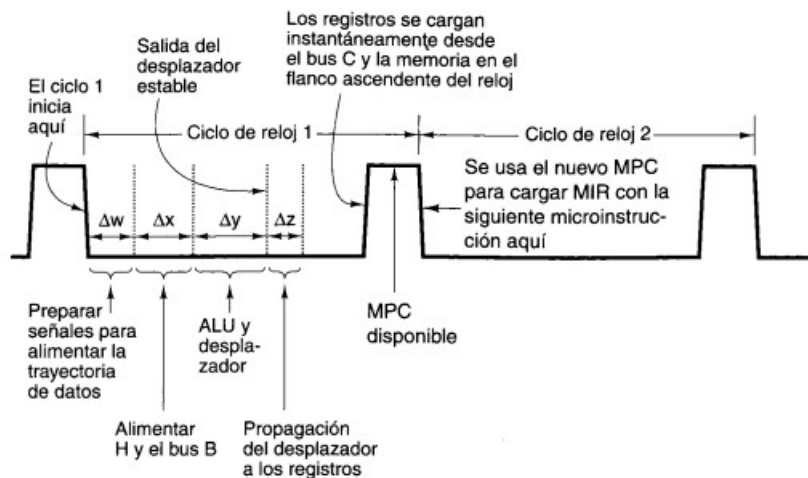


**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José

Dinámica del ciclo de reloj:

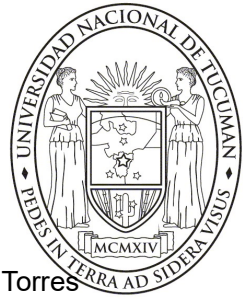
El comienzo del ciclo está definido como flanco descendente. Por esta razón en un ciclo típico, los datos de los registros en nuestro camino de datos son propagados al bus B, ALU y finalmente al bus C de la primera mitad del ciclo, ya que al llegar al flanco ascendente del ciclo, los datos del bus C son escritos en los registros apropiados. Veamos más en detalle:



En el flanco descendente de la pulsación ya están preparados los bits que alimentarán las compuertas, lo que tarda un tiempo finito conocido Δw . Luego se selecciona el registro que pasará su valor al bus B, se requiere un tiempo Δx para que el valor se estabilice. Luego la ALU y el desplazador empiezan a operar con valores válidos. Después de un tiempo adicional Δy , se estabilizan las salidas de la ALU y el desplazador. Después de otro intervalo Δz , los resultados se han propagado por el bus C hacia los registros en los que pueden cargarse durante el flanco ascendente de la siguiente pulsación. Luego se establece el MPC que define la próxima microinstrucción del próximo ciclo.

Vemos que, aunque no hay elementos de almacenamiento en la trayectoria de datos, hay un tiempo de propagación finito a través suyo. Modificar el valor que está en el bus B no hace que el bus C cambie sino hasta después de un tiempo finito. Por consiguiente, aun si una operación de almacenamiento modifica uno de los registros de entrada, el valor ya estará guardado a salvo en el registro mucho antes de que el valor (ahora incorrecto) que se está colocando en el bus B (o en H) pueda llegar a la ALU.

No hay pulsaciones de reloj ni otras señales implícitas que indiquen a la ALU cuando debe operar o que digan a los resultados que entren en el bus C. En realidad, la ALU y el desplazador están operando todo el tiempo. Sin embargo, sus entradas son basura hasta un tiempo $\Delta w + \Delta x$ después del flanco descendente del reloj. De igual manera, sus salidas son basura hasta que ha transcurrido $\Delta w + \Delta x + \Delta y$ después del flanco descendente del reloj. Las únicas señales explícitas que controlan la trayectoria de los datos son el flanco descendente y el ascendente del reloj. Si se quiere que el cargado de los registros funcione siempre, debe asegurarse de que $\Delta w + \Delta x + \Delta y + \Delta z$ termine antes del flanco ascendente del reloj, sino el registro guardará basura en lugar del resultado correcto.



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José

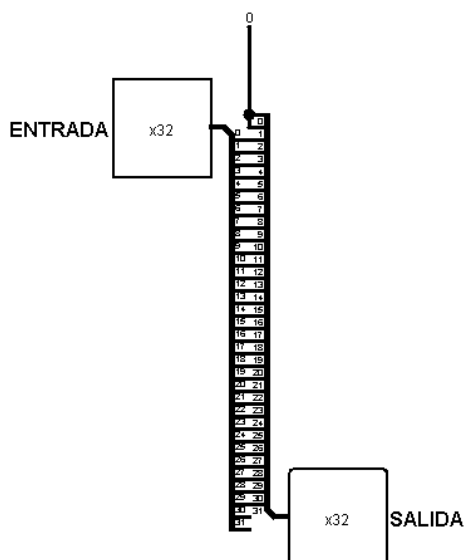
Operación de memoria:

Nuestra máquina tiene dos formas diferentes de comunicarse con la memoria: un puerto de memoria de 32 bits direccionable por palabra y otro de 8 bits direccionable por byte. El de 32 bits está bajo el control de dos registros MAR (registro de direcciones de memoria) y MDR (registro de datos de memoria). El puerto de 8 bits está bajo el control de un registro, PC, que lee un byte y lo coloca en los 8 bits de orden bajo de MBR. Este puerto solo puede leer datos de la memoria, no puede escribir datos en la memoria.

Cada uno de los registros operan con una o dos señales de control. Los que tienen una flecha blanca (encerradas con rojo en el dibujo del camino de datos), son los que pueden pasarle sus valores al bus B y los que tienen una flecha negra (encerrada con azul en el dibujo del camino de datos) pueden cargarse a partir del bus C.

MAR contiene direcciones de palabra de modo que los valores 0, 1, 2, etc. se refieren a palabras consecutivas. PC contiene direcciones de byte de modo que los valores 0, 1, 2, etc. se refieren a bytes consecutivos. Esto quiere decir que si colocamos en el MAR 0...00000, la memoria lee 0...00000, 0...00001, 0...00002, 0...00003, es decir se leen los primeros 32 bits que equivalen a una palabra y se colocan en el MDR; si coloco en el MAR 0...00001 empieza a leer la palabra desde 0...00004, si seguimos este patrón vemos que tenemos que multiplicar por 4 lo que colocamos en el MAR para saber dónde empieza la lectura en memoria de la palabra. En cambio, si colocamos 0...00001 en PC y se inicia una lectura de memoria, lee el byte 0...00001 en la memoria y se colocan los 8 bits de orden bajo de MBR.

En la MIC-1 la memoria principal está direccionada por bytes, pero el procesador opera con palabras de 32 bits, o sea 4 bytes. Entonces, las salidas de los pines del MAR se conectan al bus de direcciones desplazada dos posiciones hacia la izquierda, agregando dos ceros al final del número binario. Este desfase de dos bits multiplica el valor por cuatro, traduciendo instantáneamente y sin costo computacional la dirección de palabra del MAR a la dirección de byte de la memoria.



Desplazador del MAR, desplaza 2 bits a la izquierda-Logisim



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José

Por ejemplo si tuviéramos:

MAR	Bits desplazados a la izquierda	Dirección de memoria en bytes
0000	0000	0
0001	0100	4
0010	1000	8

En cambio el PC se conecta directamente al bus de direcciones ya que cada valor en PC representa 1 Byte de datos, no es necesario desplazar.

Esta diferencia de funcionalidad es necesaria porque MAR y PC se usarán para hacer referencia a dos partes de la memoria. La combinación MAR / MDR se usa para leer y escribir palabras de datos en el nivel ISA, y la combinación PC / MBR sirve para leer el programa ejecutable en el nivel ISA, que consiste en un flujo de bytes

Como ya se mencionó los datos leídos de memoria a través del puerto de memoria de 8 bits se devuelven en MBR, un registro de 8 bits. Hay dos formas en las que el valor del MBR puede copiarse al bus B: signada o no signada. En el caso de ser no signada, para extender el valor de 8 bits a 32 bits se agregan 24 ceros a la izquierda y si es signada, en vez de 0 se agregan todos 1 o 0 dependiendo si el bit de la izquierda del MBR es 0 o 1.

En un ciclo si una señal de lectura de memoria está habilitada, la operación de memoria se inicia al final del ciclo de la trayectoria de datos una vez que se ha cargado MAR. Los datos de memoria están disponibles hasta el final del siguiente ciclo en MBR o MDR y se usan el ciclo que sigue. Esto quiere decir que una lectura de memoria por cualquiera de los puertos, iniciada al final del ciclo k, proporciona datos que se usan en el ciclo k+2 o posteriores.

Puesto que MBR y MDR se cargan en el flanco ascendente del reloj, junto con todos los demás registros, podrían leerse durante ciclos en los que se está efectuando una nueva lectura de memoria. Esa lectura devuelve los valores antiguos ya que todavía no ha habido tiempo de sobrescribirlos.

Hex	Mnemonic	Meaning
0x10	BIPUSH <i>byte</i>	Push byte onto stack
0x59	DUP	Copy top word on stack and push onto stack
0xA7	GOTO <i>offset</i>	Unconditional branch
0x60	IADD	Pop two words from stack; push their sum
0x7E	IAND	Pop two words from stack; push Boolean AND
0x99	IFEQ <i>offset</i>	Pop word from stack and branch if it is zero
0x9B	IFLT <i>offset</i>	Pop word from stack and branch if it is less than zero
0x9F	IF_ICMPEQ <i>offset</i>	Pop two words from stack; branch if equal
0x84	IINC <i>varnum const</i>	Add a constant to a local variable
0x15	ILOAD <i>varnum</i>	Push local variable onto stack
0xB6	INVOKEVIRTUAL <i>disp</i>	Invoke a method
0x80	IOR	Pop two words from stack; push Boolean OR
0xAC	IRETURN	Return from method with integer value
0x36	ISTORE <i>varnum</i>	Pop word from stack and store in local variable
0x64	ISUB	Pop two words from stack; push their difference
0x13	LDC_W <i>index</i>	Push constant from constant pool onto stack
0x00	NOP	Do nothing
0x57	POP	Delete word on top of stack
0x5F	SWAP	Swap the two top words on the stack
0xC4	WIDE	Prefix instruction; next instruction has a 16-bit index

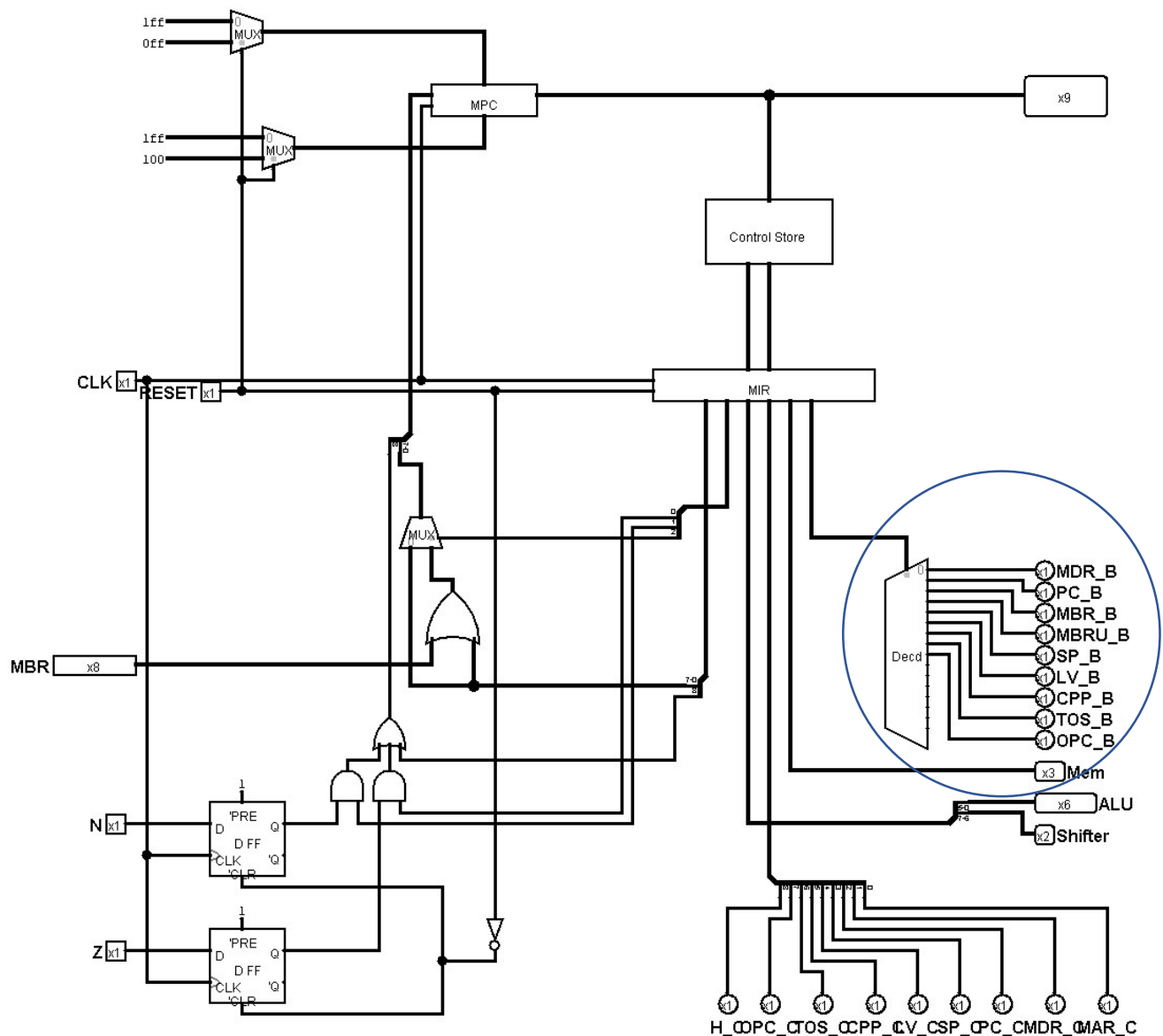


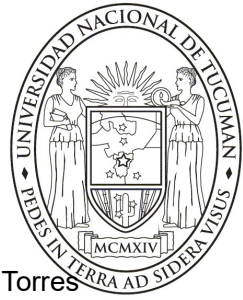
**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José

Como se puede observar en la tabla del conjunto de instrucciones IJVM, existe una variabilidad en el tamaño de las mismas. Mientras que algunas instrucciones son de un único byte (como IADD o NOP), otras están compuestas por más de un byte al requerir operandos adicionales, tal como es el caso de BIPUSH byte o GOTO offset. Esta característica expone una de las principales debilidades de la MIC-1: para poder efectuar las instrucciones de múltiples bytes, el procesador se ve obligado a realizar múltiples lecturas secuenciales a la memoria principal. Como resultado, la ALU debe permanecer inactiva durante los ciclos de espera necesarios para traer cada byte adicional, limitando severamente el rendimiento general del sistema.

UNIDAD DE CONTROL





ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS PROYECTO FINAL

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José

La unidad de control de la MIC-1 es del tipo Microprogramable. Esto significa que contiene una ROM donde sus direcciones son microinstrucciones definidas para resolver la totalidad de la instrucción cargada en MBR. Específicamente, nuestro ROM es de 512×36 bits. Es importante notar que es muy diferente de la memoria principal: las instrucciones de la memoria principal se ejecutan en orden según su dirección, las microinstrucciones no. Los microprogramas necesitan mayor flexibilidad (porque las secuencias de microinstrucciones tienden a ser cortas), así que generalmente no tienen esta propiedad. En lugar de esto, cada microinstrucción especifica explícitamente su sucesora.

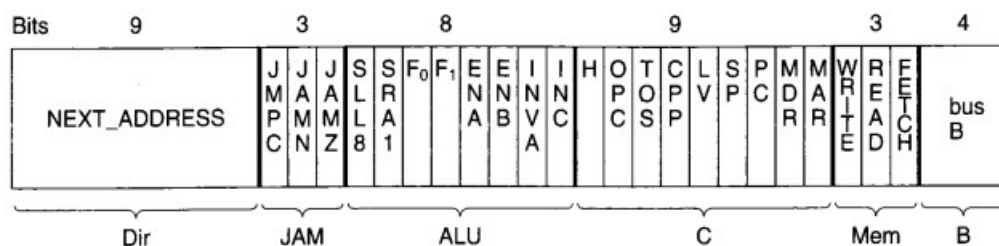
Puesto que la unidad de control es funcionalmente una memoria, necesita su propio registro de direcciones de memoria y su propio registro de datos de memoria. No necesita señales de lectura, porque continuamente se está leyendo. Llamamos al registro de dirección de memoria de 9 bits MPC (Microprogram Counter) y al registro de datos de memoria de 36 bits MIR (Microinstruction register) cuya función es almacenar la microinstrucción actual.

Existen 29 señales de control que gestionan como funciona el camino de datos durante un ciclo de reloj. Estas pueden ser divididas en 3 grupos, marcadas en la pagina 1:

- 9 señales para escribir datos del bus C a los registros (Azules)
- 9 señales para escribir datos de registros al bus B (Rojas)
- 6 para controlar el ALU (Naranjas) y 2 para el shifter (Verdes)
- 2 señales (que no se muestran) para indicar lectura/escritura de memoria vía MAR / MDR
- 1 señal (que no se muestra) para indicar obtención de memoria vía PC / MBR

Para alimentar al bus B se utilizan 9 señales de control. Como hay 9 registros, podemos codificar la información del bus B en 4 bits usando un decodificador (marcado con azul) para seleccionar cada una de ellas generando 16 combinaciones distintas para las 9 señales de control, 7 de las cuales no se utilizan. Para la MIC-1 se utilizó un decodificador estándar 4 a 16 y no uno 4 a 9 ya que utilizar un circuito estándar es más fácil y reduce la posibilidad de errores, en cambio si usáramos uno de 4 a 9, aunque ocuparía menos área de chip, sería más difícil y habría mayor probabilidad de error.

Luego, podemos controlar la trayectoria de datos con 24 señales y por tanto 24 bits. Pero estos 24 bits sólo controlan el camino de datos durante un ciclo. La segunda parte del camino de datos consiste en determinar la siguiente microinstrucción que está gobernada por dos campos del MIR: NEXT_ADRESS (campo de 9 bits que contienen una dirección potencial de la microinstrucción) y JAM (define como la próxima microinstrucción es seleccionada).





**ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL**

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José

- Dir (9 bits): Contienen una dirección potencial de la próxima microinstrucción
- JAM (3 bits): Define como se selecciona la próxima microinstrucción
- ALU (8 bits) : Señales para el ALU y shifter
- C (9 bits): Señales para el bus C
- Mem (3 bits): Señales para operaciones de memoria
- B (4 bits): Señales para el bus B

Como explicamos en la dinámica del ciclo de reloj, la ejecución de la microinstrucción se divide en cuatro subciclos. En el primero (Δw) se carga el MIR; en el segundo (Δx) se vuelcan los datos al Bus B; en el tercero (Δy) operan la ALU y el desplazador; y finalmente, en el cuarto (Δz), los resultados viajan por el Bus C para ser guardados en los registros junto con los de estado N y Z. Esto termina un poco después del flanco ascendente del reloj, cuando todos los resultados se han guardado y los resultados de las operaciones de memoria anteriores están disponibles, y se ha cargado MPC. Este proceso se repite mientras el procesador se mantenga en funcionamiento.

Al mismo tiempo que controla la trayectoria de datos, el microprograma tiene que determinar qué microinstrucción ejecutará a continuación. El cálculo de la dirección de la siguiente microinstrucción se inicia una vez que MIR se ha cargado y estabilizado. El valor del MPC puede estar compuesto por el valor de MBR, Addr del MIR, N, y Z, dependiendo de los valores de JAM del MIR. Las dos posibilidades son las siguientes:

- Si JMPC es 1, las funciones booleanas son
$$\text{MPC}[7-0] = \text{MBR}[7-0] + \text{NEXT_ADDRESS}[7-0]$$
y
$$\text{MPC}[8] = \text{NEXT_ADDRESS}[8]$$
- Si JMPC es 0, las funciones booleanas son
$$\text{MPC}[7-0] = \text{NEXT_ADDRESS}[7-0]$$
y
$$\text{MPC}[8] = (\text{JAMZ} \cdot \text{Z}) + (\text{JAMN} \cdot \text{N}) + (\text{NEXT_ADDRESS}[8])$$

Registros N y Z son registros de 1-bit que se escriben en el flanco ascendente. Estos dos registros se conocen como registros de estado. N es 1 cuando el resultado del ALU es negativo, mientras Z es 1 cuando el resultado del ALU es cero. N y Z son de suma importancia cuando estamos buscando hacer saltos con las instrucciones IFEQ offset, IFLT offset y IF_ICMPEQ offset.



ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORAS
PROYECTO FINAL

PARTICIPANTES: Pedraza Francisco Benjamín, Ponce Lucía Isabel, Selis Octavio, Torres Matias José

CONCLUSION:

A lo largo de este trabajo se logró diseñar, simular y analizar de manera integral la microarquitectura MIC-1 mediante la herramienta Logisim. Sin embargo, como dijimos anteriormente, el diseño de la MIC-1 prioriza la claridad pedagógica por encima de la eficiencia operacional, lo que deriva en importantes deficiencias de rendimiento. Empezando por la dependencia del registro H: al contar con un único bus de entrada a la ALU (Bus B), la máquina se ve obligada a utilizar el registro H como almacenamiento temporal obligatorio para el primer operando. Esto exige gastar ciclos de reloj adicionales simplemente en preparar los datos antes de poder efectuar una operación aritmética o lógica. También la división del ciclo en cuatro subciclos limita la frecuencia de reloj máxima y desperdicia tiempo en operaciones que no requieren los cuatro pasos. Además cada vez que el procesador necesita leer un byte de instrucción o una palabra de datos de la memoria principal, debe entrar en ciclos de espera, congelando el Camino de Datos hasta que la memoria responda.

Por estas razones, en el libro, Tanenbaum propone una serie de microarquitecturas (MIC-2, MIC-3, MIC-4) que presentan mejoras en estos aspectos y vale la pena indagar. La MIC-2 agrega una Unidad de Prebúsqueda de Instrucciones (IFU - Instruction Fetch Unit) que lee los bytes de la memoria por adelantado y los carga en registros (MBR1, MBR2), para que cuando la ALU los necesite, ya estén ahí. La MIC-3 conserva la IFU pero además divide la trayectoria de datos en partes discretas usando latches que permiten que cada parte trabaje de forma independiente y acelerar el reloj. Por último la MIC-4 en lugar de una sola ALU, la MIC-4 tiene varias ALUs, varios desplazadores y varias unidades de memoria trabajando en paralelo, por lo que puede ejecutar más de una microinstrucción por ciclo de reloj.

En fin, la experiencia de simular la MIC-1 en Logisim y analizar este microprocesador nos permitió comprender en profundidad las bases de la arquitectura de las computadoras.